

Operating Systems 2024

U3680
National Taipei University



Final Exam [Chapter 6 - Chapter 10]

1. Select **all correct** statements about **synchronization** (8%)

- (a) The **naive solution** (using a shared variable, **turn**, to synchronize two processes) satisfies the **progress** property
- (b) **Peterson's solution** satisfies the **bounded waiting** property
- (c) **Atomic swap** satisfies the **bounded waiting** property
- (d) **Bakery's algorithm** satisfies the **progress** and **bounded waiting** properties

(b) and (d)

2. The following two processes, `action()` and `counter()`, are synchronized using condition variables. Explain why the `pthread_cond_wait` function needs a **mutex** as a parameter (give an example that the synchronization will fail without the mutex) (6%)

```
action() {  
    pthread_mutex_lock(&mutex);  
    if (x != 0)  
        pthread_cond_wait(cond, mutex);  
    pthread_mutex_unlock(&mutex);  
    take_action();  
}
```

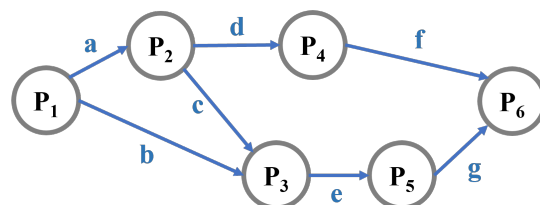
```
counter() {  
    pthread_mutex_lock(&mutex);  
    x-;  
    if (x == 0)  
        pthread_cond_signal(cond);  
    pthread_mutex_unlock(&mutex);  
}
```

If the variable x is not equal to zero and the action executes before the counter, it will hold the lock. After it enters the critical section (CS), it will be put to **sleep** by the `pthread_cond_wait` function because x is not zero. At this point, the counter will be blocked outside the CS since it cannot acquire the lock, resulting in a **deadlock**. For the above reason, we need to pass the lock into the function as a parameter for letting OS release the lock.

3. In the below figure, each edge represents a dependency between two processes (e.g., $P_1 \rightarrow P_2$ means P_1 must execute before P_2). Please fill in the operations needed for each process using **wait**, **operation S_i for P_i** , and **signal** with the semaphore variables on the edges (6%)

P_1 : _____
 P_2 : _____
 P_3 : _____
 P_4 : _____
 P_5 : _____
 P_6 : _____

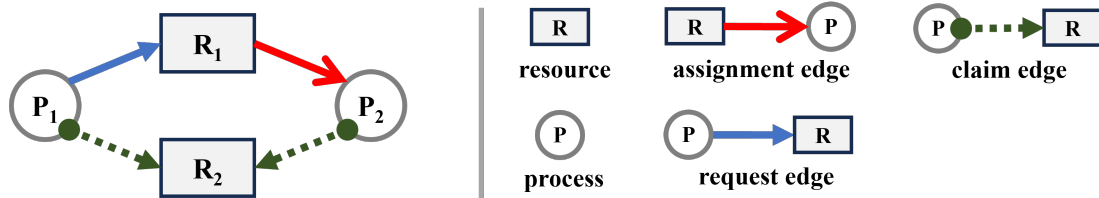
using S_i ,
wait (EDGE), and
signal (EDGE)



P_1 : S_1 ; signal(a); signal(b);

P_2 : wait(a); S_2 ; signal(c); signal(d);
 P_3 : wait(b); wait(c); S_3 ; signal(e);
 P_4 : wait(d); S_4 ; signal(f);
 P_5 : wait(e); S_5 ; signal(g);
 P_6 : wait(f); wait(g); S_6 ;

4. Perform **deadlock avoidance** based on the following resource-allocation graph:



(a) If P_1 requests R_2 , should OS accept? why? (4%)

No. Assigning R_2 to P_1 might result in a circle in the future (when P_2 requests R_2).

(b) If P_2 requests R_2 , should OS accept? why? (4%)

Yes. Assigning R_2 to P_2 will not result in a circle (P_1 can get R_2 after P_2 has finished).

5. In **First Readers-Writers problem**, no reader will be kept waiting unless a writer is updating the shared data. Explain how we can synchronize the writer/readers using **semaphores**. (6%)

We can use two semaphores for synchronization. The first one, wrt, synchronizes the writers and the first reader, and the second one, mutex, updates the number of readers. The last reader has to release the wrt lock for the writer.

6. Please give a method to prevent the **circular wait** condition of **deadlock** while designing a system. **You should prove why it works** (6%)

(a) We can impose a total ordering of all resource types. A process should request resources in increasing order.

(b) **Proof:** Let $P_i(R_j) \rightarrow R_k$ means process i holds resource j and waits for resource k :

$P_0(R_0) \rightarrow R_1, P_1(R_1) \rightarrow R_2, \dots, P_n(R_n) \rightarrow R_0$

Conflict with $R_0 < R_1 < R_2 < \dots < R_n < R_0$

7. Consider the following snapshot of a system. The total resource (A, B, C, D) = (5, 7, 5, 7). Using **Banker's algorithm** to answer the following questions:

(a) Is the current state safe? If yes, illustrate the order in which the processes may be completed. Otherwise, give the reason (4%)

Yes. The processes can be completed in order: $P_2 \rightarrow P_3 \rightarrow P_1 \rightarrow P_4$

(b) If process P_4 requests an additional A, should OS allocate the resource? Why? (4%)

Yes. The processes still can be completed in order: $P_2 \rightarrow P_3 \rightarrow P_1 \rightarrow P_4$

	Allocation					Max			
	A	B	C	D		A	B	C	D
P_1	2	3	2	3		3	4	4	6
P_2	1	2	0	1		1	2	2	1
P_3	1	1	0	1		2	2	2	1
P_4	0	1	1	1		2	2	4	3

8. Assume a system allows a process to have at most **512** pages and uses a **single-level page table**. The page size is **2 KB**. The physical memory is **16 MB**. Please answer the following questions:
- What is the **logical address** for **page number 29** and **offset 40**? (4%)
00001110100000101000
 - Assume that **page 29** maps to **frame 15**, what is the **physical address** in (a)? (4%)
000000000111100000101000
9. Assume each memory read takes **200** nanoseconds and each TLB search takes **20** nanoseconds in a **single-level paging** system. Answer the following questions:
- What is the effective memory reference time if the TLB hit ratio is 80%? (4%)
 $0.8 \times 220 + 0.2 \times 420 = 260$
 - What is the necessary TLB hit rate if we need the effective memory reference time smaller than **240** nanoseconds? (4%)
90%
10. Apply the (1) **First-In-First-Out (FIFO)** and (2) **Least Recently Used (LRU)** algorithms for page replacement with the page-reference strings: (1, 2, 1, 4, 5, 1, 2, 1). Find the number of page faults for each algorithm assuming demand paging with **three** frames. **You need to show the progress of each step.** (8%)
FIFO: 6 page faults; LRU: 5 page faults

	1	2	1	4	5	1	2	1
FIFO	1	2	2	4	5	1	2	2
		1	1	2	4	5	1	1
				1	2	4	5	5
LRU	1	2	1	4	5	1	2	1
		1	2	1	4	5	1	2
			2	1	4	5	5	5

11. Please answer the following questions about **thrashing**:
- Describe why the CPU utilization in a system drops so fast when a thrashing occurs. (6%)
The CPU utilization drops due to the allocated frames for a process are not enough, resulting in more I/O → OS detects the drop in CPU utilization and brings more processes to improve CPU utilization → New processes take frames from old processes, resulting in more page faults → The CPU utilization drops even further
 - Describe how **Working-Set Model** and **Page Fault Frequency Scheme** are used to avoid thrashing while maintaining system utilization, respectively. (8%)
 - Working-Set Model:** record the set of **referenced pages within a time window** for every process (called working-set size). If the sum of the working-set size of all processes is larger than the available frames, suspend a process (thrashing occurs). Otherwise, increase the degree of multi-programming (4%)

- **Page Fault Frequency Scheme:** establish upper and lower bounds on the desired page-fault rate of a process. If the page fault rate exceeds the upper limit, allocate another frame to the process. If the page fault rate falls below the lower limit, remove a frame from the process (4%)
- (c) Why **Page Fault Frequency Scheme** is cheaper than **Working-Set Model**? (4%)
Working-Set Model needs to keep track of **every page reference for every process**, which is expensive. In contrast, **Page Fault Frequency Scheme** only updates the count when page faults occur, which is **less frequent**
12. Select **all correct** statements about **main memory** (10%)
- (a) A process that is doing I/O can be swapped out to disk for saving memory storage
 - (b) Using **load-time address binding** results in the same logical address and physical address
 - (c) **Dynamic loading** can avoid duplicate loading of the same libraries
 - (d) **Hierarchical paging** is the most popular choice in modern 64-bit systems
 - (e) One entry in the **inverted page table (frame table)** requires to store a pair of PID and page number
- (b) and (e) or (a), (b), and (e)